

Distributed Web Crawling and Parallel Page Ranking

Hamza Ahmed

h.ahmed.29265@khi.iba.edu.pk

Karachi, Pakistan

Muhammad Hamza Arif

m.arif.29234@khi.iba.edu.pk

Karachi, Pakistan

Laraib Fatima Farooqui

l.masood.26521@khi.iba.edu.pk

Karachi, Pakistan

Arbaaz Murtaza

a.murtaza.29052@khi.iba.edu.pk

Karachi, Pakistan

Abstract—This paper presents the design, implementation, and empirical evaluation of a three-milestone parallel and distributed computing project centred on web-scale PageRank computation. Milestone 1 builds and benchmarks three web crawling architectures—Sequential, Ray Master-Worker, and Ray Peer-to-Peer—across a hyperparameter grid of seed counts (10, 20, 40), crawl depths (1–3), and worker counts (4–12). Master-Worker achieves up to 425% throughput improvement over Sequential at large workloads, while P2P reaches 128% at 12 workers. Milestone 2 applies parallel PageRank to the crawled graphs (54K–1.5M nodes) using two Ray-based strategies: Centralised Aggregation (CA) and Distributed Reduction (DR). CSR format reduces memory by 20–30×; DR with convergence-based termination achieves up to 2.70× speedup on the largest graph. All experiments were conducted on a single machine using Ray for multiprocessing. True distributed performance across multiple nodes was not evaluated. Milestone 3 extends the engine to adaptive incremental updates using a 2-Stage MapReduce Edge-Partitioning architecture. A single-page micro-update executes in $\sim 20 \mu\text{s}$ vs. 24 s for full recomputation—a 1.57-million-fold reduction in nodes processed—while a 50K-node MapReduce batch completes in 14.11 s, 1.7× faster than full recomputation at Depth 3.

Index Terms—distributed web crawling, PageRank, task parallelism, Ray framework, incremental graph updates, edge partitioning, MapReduce, parallel computing

I. INTRODUCTION

Large-scale web graph construction is a foundational step in link-analysis algorithms such as PageRank [1]. Crawling the web, however, is inherently I/O-bound: each URL fetch involves a network round trip whose latency dominates compute cost. Exploiting parallelism—both at the concurrency level (overlapping I/O waits) and at the distribution level (multiple independent workers)—is therefore essential for achieving practical throughput.

This project is structured around three milestones within a parallel and distributed computing (PDC) curriculum. Milestone 1 (reported here) focuses on building and benchmarking the data-ingestion pipeline that constructs the web graph. Milestone 2 will apply a parallel PageRank algorithm to this graph, and Milestone 3 will extend the system to support incremental graph updates.

The central research questions for Milestone 1 are:

- 1) Under what workload conditions does Ray-based parallelism outperform a single-process asynchronous crawler?

- 2) How do centralised (Master-Worker) and decentralised (Peer-to-Peer) coordination strategies compare in throughput and scalability?
- 3) How do hyperparameters—seed count, crawl depth, and worker count—interact to affect system performance?

To answer these questions, all three architectures are implemented using the Ray distributed runtime [2] and the aiohttp asynchronous HTTP library, and are subjected to a controlled hyperparameter sweep. Seed URLs are sampled reproducibly from the Majestic Million top .com domains [3]. The decentralised Peer-to-Peer architecture is inspired by prior work on distributed web crawling [4].

The remainder of this paper is organised as follows. Section II provides background on asynchronous crawling and the Ray framework. Section III describes the code architecture shared by all three implementations. Sections IV–VI detail each crawler. Section VII defines the experimental setup. Section VIII presents quantitative results. Section IX discusses key findings, and Section X concludes with implications for Milestone 2.

Milestone 1: Web crawling

II. BACKGROUND

A. Asynchronous I/O for Web Crawling

Python’s `asyncio` event loop enables cooperative multitasking within a single OS thread by suspending coroutines at `await` points (typically network I/O) and resuming them when data arrives. This *concurrency* model avoids the thread-creation and context-switching overheads of thread-pool designs. Prior to introducing Ray, an `asyncio`-only crawler (`Milestone1_Asyncio.py` and its improved variant `Milestone1_Asyncio_Mod.py`) was developed to validate link extraction logic, per-domain rate limiting, and crawl-frontier management on a single machine.

B. Ray: Task-Based Distributed Runtime

Ray [2] provides a unified API for task parallelism and actor-based stateful computation across a cluster. A Ray *actor* is a long-lived Python object executed in a dedicated worker

process; remote method calls are non-blocking and return futures. Ray’s object store allows workers to share large data with minimal serialisation overhead. These properties make Ray well suited to the web-crawling domain, where workers must coordinate on a shared frontier while performing high-latency network fetches.

III. SYSTEM ARCHITECTURE

A. Shared Utilities

All three crawlers rely on a common utility layer that ensures consistent behaviour across architectures:

- **normalize_url**: Strips `www`, lowercases the network location, removes trailing slashes, and drops query strings and fragments to eliminate duplicate URL variants.
- **get_base_domain**: Extracts the registered domain from a URL for per-domain rate limiting.
- **hash_id**: Computes an MD5-based consistent hash to assign a URL to a specific worker partition (used exclusively by P2P).
- **make_ssl_ctx**: Returns an SSL context with certificate verification disabled, improving crawl robustness.
- **load_seeds**: Samples n `.com` domains from the Majestic Million CSV with `random_state=42` for reproducibility.

B. Graph Representation and Output

Each crawler constructs a directed graph where nodes represent normalised URLs and edges represent hyperlinks. The graph is stored as a JSON adjacency list (`crawl_graph.json`) upon crawl completion. This format is directly suitable for partitioned parallel PageRank computation in Milestone 2.

IV. SEQUENTIAL CRAWLER

The `SequentialCrawler` class implements a breadth-first search (BFS) crawler entirely within a single Python process. It uses `aiohttp` for asynchronous HTTP requests with a 10-second timeout and `selectolax` for fast HTML parsing. A per-domain cap (`max_per_domain=50`) prevents exhaustive crawling of any single site. URLs are processed in concurrent batches of 50 via `asyncio.gather`, and all state (`visited`, `domain_counts`, `graph`) is maintained in plain Python data structures.

Although the crawler is called “sequential”, it exploits I/O concurrency through `asyncio`; it is *sequential* in the sense that it uses a single OS process and no distributed workers.

V. RAY MASTER-WORKER CRAWLER

A. Design

The Master-Worker architecture decomposes the crawl into two actor types:

- **MWMaster**: A centralised Ray actor that owns all shared state—the BFS queue, the visited URL set, the graph, and per-domain counts. It exposes two remote methods: `get_batch()` (distributes URL batches to workers) and `submit_results()` (ingests discovered links,

deduplicates, and re-enqueues within-depth-limit children).

- **MWWorker**: Stateless fetch actors that loop continuously: pull a batch from the master, fetch each URL asynchronously, extract outgoing links, and submit results back. Workers use `max_concurrency=2` to allow overlapping fetch cycles.

B. Trade-offs

Centralising all state in `MWMaster` eliminates race conditions and guarantees that no URL is visited twice. However, the master actor serialises all state updates, which becomes a bottleneck as the number of workers grows. This explains the diminishing returns observed when scaling from 8 to 12 workers (Section VIII).

VI. RAY PEER-TO-PEER CRAWLER

A. Design

The Peer-to-Peer architecture removes the central coordinator. Each `P2PWorker` actor is assigned a disjoint partition of the URL space via consistent hashing (`hash_id`). When a worker discovers a new link, it computes that link’s owning worker and routes it there via a direct Ray remote call (`receive()`), bypassing any global queue.

Termination is managed by a lightweight `P2PSignal` actor: when any worker’s queue is exhausted, it initiates a countdown timer (120 s); once the timer elapses, all workers check the signal and halt. Per-domain concurrency is regulated by an `asyncio.Semaphore(3)` within each worker, and each actor is configured with `max_concurrency=100` to support high async throughput.

B. Trade-offs

Eliminating the central actor removes the serialisation bottleneck, enabling better scaling at high worker counts. However, hash-based partitioning can leave some workers with empty or very sparse queues at low worker counts, causing a significant fraction of seeds to go unvisited. This is the primary reason for the lower URLs-crawled counts observed in P2P at 4 workers (Section VIII).

VII. EXPERIMENTAL SETUP

A. Dataset

Seed URLs were sampled from the Majestic Million `.com` domain list using a fixed random state (42) to ensure reproducibility across runs.

B. Hyperparameter Grid

Each crawler was evaluated over the full grid shown in Table I. Ray-based crawlers were additionally swept over worker counts. This yields 9 configurations for Sequential and 27 per Ray architecture.

TABLE I: Hyperparameter Grid

Parameter	Values	Applies To
num_seeds	10, 20, 40	All
depth	1, 2, 3	All
num_workers	4, 8, 12	Ray only

C. Metrics

Four metrics are recorded per configuration:

- **Elapsed time** (seconds): wall-clock duration from start to finish.
- **URLs crawled**: total unique URLs successfully fetched.
- **Graph edges**: directed hyperlinks in the crawled graph.
- **Throughput** (URLs/s): $\text{urls_crawled} / \text{elapsed_sec}$, the primary performance indicator.

VIII. RESULTS

A. Sequential Crawler

Table II summarises results for the Sequential crawler. Throughput remains stable between 8 and 15 URLs/s across all configurations, confirming that a single-process design imposes a hard ceiling regardless of workload size. The 20-seed, depth-1 configuration yields the highest throughput at 15.2 URLs/s. At depth 3 and 40 seeds, elapsed time reaches 2,153 s (≈ 36 minutes), while graph edges scale to 814,048, illustrating the exponential growth of the crawl frontier with depth.

TABLE II: Sequential Crawler Results

Seeds	Depth	Time (s)	URLs	Edges	URLs/s
10	1	14.8	127	4,615	8.6
10	2	43.4	457	10,888	10.5
10	3	98.7	1,062	27,111	10.8
20	1	23.3	353	16,921	15.2
20	2	77.4	1,018	32,233	13.2
20	3	258.3	3,367	99,169	13.0
40	1	117.8	1,035	45,481	8.8
40	2	487.1	4,158	162,588	8.5
40	3	2,153.7	19,268	814,048	8.9

B. Ray Master-Worker Crawler

Selected results for the Master-Worker crawler are given in Table III. At low workloads (10 seeds, depth 1), the Ray actor initialisation overhead dominates, and all worker counts underperform Sequential (up to 61% slower at 4 workers). At higher workloads the situation reverses dramatically: with 40 seeds, depth 2, and 12 workers, throughput reaches 44.8 URLs/s—a 425% improvement and a $5.1\times$ speedup over the Sequential baseline (93.5 s vs. 487.1 s). The largest single jump in throughput occurs when scaling from 4 to 8 workers; going from 8 to 12 yields diminishing returns as the central master actor becomes a serialisation bottleneck.

TABLE III: Ray Master-Worker Results (selected configurations)

S	D	W	Time (s)	URLs	URLs/s	vs Seq
10	1	4	37.7	127	3.4	−61%
10	1	12	27.0	127	4.7	−46%
10	2	8	36.6	451	12.3	+17%
10	2	12	35.6	451	12.7	+21%
20	2	8	35.0	1,026	29.3	+122%
20	2	12	35.7	1,015	28.5	+116%
40	2	8	141.8	4,094	28.9	+239%
40	2	12	93.5	4,189	44.8	+425%

S = Seeds, D = Depth, W = Workers

C. Ray Peer-to-Peer Crawler

Selected P2P results appear in Table IV. P2P performance is markedly weaker at low worker counts: with 4 workers and 10 seeds at depth 1, only 10 URLs are crawled in 11.2 s (0.9 URLs/s), a 90% regression from Sequential. This is caused by hash-based partitioning: at 4 workers, many partitions receive few or no seeds, leaving those workers idle. Performance improves substantially as the worker count rises to 12, where P2P becomes competitive with Master-Worker. The best P2P configuration (20 seeds, depth 2, 12 workers) achieves 30.0 URLs/s (+128% over Sequential), slightly exceeding the Master-Worker at the same setting (28.5 URLs/s).

TABLE IV: Ray Peer-to-Peer Results (selected configurations)

S	D	W	Time (s)	URLs	URLs/s	vs Seq
10	1	4	11.2	10	0.9	−90%
10	1	12	14.5	72	5.0	−42%
10	2	12	29.1	415	14.3	+36%
20	1	4	13.7	25	1.8	−88%
20	2	8	32.9	561	17.1	+30%
20	2	12	30.3	908	30.0	+128%
40	2	4	301.1	1,905	6.3	−26%
40	2	12	184.2	2,321	12.6	+48%

S = Seeds, D = Depth, W = Workers

IX. DISCUSSION

A. Workload Threshold for Parallelism

A consistent finding across both Ray architectures is that parallelism is only beneficial above a certain workload threshold. At depth 1 with 10–20 seeds, Ray actor startup and inter-actor message passing costs exceed the time saved by concurrent fetching. This is a well-known property of distributed systems: the fixed overhead of coordination must be amortised over sufficiently large computational tasks [5].

B. Centralised vs. Decentralised Coordination

Master-Worker and Peer-to-Peer represent two ends of a coordination spectrum:

- **Correctness**: Master-Worker’s global visited set guarantees that every URL is processed at most once. P2P relies on per-worker visited sets plus hash-based ownership,

which can cause some URLs to go unvisited when the hash distribution is uneven at low worker counts.

- **Scalability:** P2P eliminates the master serialisation bottleneck and thus scales better in principle. However, the benefit only materialises at 12 workers in these experiments; below that threshold, the lower URL-coverage of P2P more than offsets any latency advantage.
- **Throughput ceiling:** At 12 workers and medium depth, P2P slightly exceeds Master-Worker throughput (30.0 vs. 28.5 URLs/s at 20 seeds, depth 2). This suggests that at larger cluster sizes, P2P could become the preferred architecture.

C. Worker Scaling

In both Ray architectures, the 4→8 worker transition yields the largest per-step throughput gain (roughly doubling in most configurations), while the 8→12 transition shows diminishing returns. For Master-Worker, this is attributable to master actor saturation. For P2P, the gain is limited by the fixed number of seed URLs: with 12 workers and only 10 seeds, most workers remain idle regardless of coordination strategy.

D. Graph Scale

Graph edge counts grow super-linearly with depth. Moving from depth 2 to depth 3 at 40 seeds increases edges from $\approx 162,000$ to $\approx 814,000$ —a roughly $5\times$ multiplier. This confirms that depth is the dominant driver of graph size and, consequently, of downstream PageRank computation cost. Workload planning for Milestone 2 should account for this scaling behaviour when selecting the input graph.

E. Architecture Recommendation

Based on the Milestone 1 findings:

- For large-scale crawls (≥ 40 seeds, depth ≥ 2): Ray Master-Worker with 8–12 workers is the recommended architecture due to its consistent correctness and highest absolute throughput (44.8 URLs/s).
- For small or shallow crawls (≤ 20 seeds, depth 1): the Sequential crawler remains competitive and avoids distributed overhead.
- For future cluster deployments with many workers: Ray P2P offers better scaling potential, provided consistent-hash partition skew is mitigated (e.g., via virtual nodes or adaptive load balancing).

X. CONCLUSION

Milestone 1 successfully delivers three web crawling architectures—Sequential, Ray Master-Worker, and Ray Peer-to-Peer—evaluated across a systematic hyperparameter sweep. The key takeaways are:

- 1) Ray-based parallelism outperforms sequential crawling only at sufficiently large workloads (depth ≥ 2 and seeds ≥ 20).
- 2) Master-Worker is the most reliable architecture, achieving a $5.1\times$ speedup at the heaviest configuration with no URL-coverage losses.

- 3) Peer-to-Peer scales better in principle but suffers from partition imbalance at low worker counts, causing missed URLs and lower throughput.
- 4) Graph edge counts scale aggressively with crawl depth, a critical consideration for Milestone 2 PageRank computation.

All graph outputs are stored as JSON adjacency lists and are ready for partitioned parallel PageRank in Milestone 2, where the PDC focus will shift to data partitioning, synchronisation barriers, and iterative parallel algorithms.

Milestone 2: Parallel PageRank Computation

XI. PAGERANK: ALGORITHM AND THEORY

A. The Core Idea

PageRank [1] answers a fundamental question in link analysis: which pages on the web are most important? The key insight is that a page’s importance is determined not only by how many pages link to it, but by the importance of those linking pages. A link from a highly ranked page contributes more value than a link from an obscure one.

B. The Random Surfer Model

PageRank is most intuitively understood through the *random surfer* model. A user browses the web and at each step either follows a random link on the current page with probability $d = 0.85$, or teleports to a uniformly random page with probability $1 - d = 0.15$. The PageRank of a page is simply the long-run fraction of time the random surfer spends on it. Pages receiving links from many important pages will be visited more often, earning a higher rank.

C. The Formula

For a page u with inbound link set B_u , PageRank is defined as:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in B_u} \frac{PR(v)}{L(v)} \quad (1)$$

where N is the total number of nodes, $d = 0.85$ is the damping factor, and $L(v)$ is the out-degree of node v . The teleportation term $\frac{1-d}{N}$ ensures every node retains a nonzero baseline probability of being visited even with no inbound links.

D. Iterative Computation and Convergence

Because Eq. (1) is circular, it is solved iteratively: (i) initialise every node with rank $\frac{1}{N}$; (ii) apply Eq. (1) to all nodes simultaneously using ranks from the previous round; (iii) repeat until the L_∞ norm of rank change falls below a tolerance $\varepsilon = 10^{-6}$, or a fixed iteration cap is reached. Convergence is guaranteed because PageRank is a contraction mapping on the probability simplex; in practice it occurs within 7–20 iterations for large web graphs, depending on graph structure and seed count.

E. Dangling Nodes

A *dangling node* has no outbound links. In the datasets used here, over 90% of nodes are dangling (Table VI), consistent with real web crawls in which most pages link only to external domains not captured by the crawl. Our implementation handles dangling nodes implicitly: they contribute zero rank to any successor, and their probability mass is absorbed into the uniform teleportation baseline $\frac{1-d}{N}$.

XII. PAGERANK IMPLEMENTATION

A. Graph Representation: From JSON to CSR

The crawl graph produced by Milestone 1 is a JSON dictionary mapping each URL to its list of outbound URLs. At scale, storing this as Python dicts and lists is prohibitively expensive for three reasons: (i) Python dictionaries allocate sparse hash tables at 66% load factor; (ii) every integer is boxed as a `PyLongObject` (28 bytes vs. 4 bytes in C); and (iii) Python lists store scattered object pointers, destroying CPU cache locality.

The solution is *Compressed Sparse Row* (CSR) format—three flat NumPy integer arrays:

- **indices**: concatenated inbound neighbour IDs for every node.
- **indptr**: offset array where `indptr[i]:indptr[i+1]` slices out node i 's neighbours.
- **out_counts**: outbound-link count per node, used to normalise rank contributions.

The memory reduction is substantial. A 500 MB JSON graph that would occupy 2.5–5 GB as Python dicts and lists shrinks to just 80–150 MB in CSR format—a 20–30 $\times$ reduction that makes processing 1.5M nodes on 8 GB RAM feasible. After CSR construction, the raw Python dictionary is explicitly freed and garbage-collected. Observed CSR build times are 0.15 s (Graph 1), 1.51 s (Graph 2), and 7.77 s (Graph 3).

B. Sequential PageRank

The sequential baseline is a direct NumPy translation of Eq. (1). Key design decisions are: (i) *double buffering*—two arrays `pagerank` and `new_ranks` are swapped after each full sweep to prevent partially-updated values from contaminating the current iteration; (ii) *vectorised inner sum*—`pagerank[inbound]` fetches all neighbour ranks as a NumPy slice and `np.sum()` computes the contribution in a single C-level loop; and (iii) *early stopping*—the convergence check computes the maximum absolute change after each sweep and halts as soon as it drops below 10^{-6} .

C. Parallel Implementations

Both parallel strategies share the same CSR data structure, double-buffer logic, and convergence criterion. They differ only in how per-iteration results are aggregated.

Centralised Aggregation (CA). Nodes are split into 12 equal-sized chunks, one per Ray worker. The CSR arrays

are pushed into the Ray object store once via `ray.put()` for zero-copy reads. The current rank vector is written to a memory-mapped file (`numpy.memmap`) so workers can read it without serialisation overhead. After each iteration, the driver collects all chunk results via `ray.get()` and writes the merged vector back. This incurs $\mathcal{O}(\text{num_workers})$ `ray.get()` calls per iteration, making the driver a serial bottleneck.

Distributed Reduction (DR). Workers compute their chunks identically to CA, but merge results through a binary tree of Ray tasks rather than returning to the driver. The tree depth is $\lceil \log_2(\text{num_workers} + 1) \rceil$; for 12 workers, this requires 4 reduction rounds. Only *one* `ray.get()` per iteration is needed (the merged root), reducing communication from $\mathcal{O}(\text{num_workers})$ to $\mathcal{O}(\log(\text{num_workers}))$. The trade-off is additional task-scheduling overhead from the extra Ray reduction tasks.

XIII. EXPERIMENTAL SETUP — MILESTONE 2

A. Hardware

All experiments were run on a single machine with the configuration in Table V. Ray was configured with 12 workers—one per logical core. *All experiments were conducted on a single machine using Ray for multiprocessing. True distributed performance across multiple nodes was not evaluated.*

TABLE V: Hardware Configuration

Component	Specification
Processor	AMD Ryzen 5 7430U (6c / 12t)
Memory	8 GB DDR4
Storage	256 GB SSD
Framework	Ray + NumPy 2.4.3 + Pandas 3.0.1
Ray workers	12 (one per logical core)

B. Termination Policies

Every algorithm is evaluated under two stopping conditions:

- **Fixed iteration count**: always runs exactly 25 iterations.
- **Convergence-based**: stops when $\|\Delta PR\|_\infty < 10^{-6}$ (L_∞ norm).

C. Metrics

For each configuration the following are recorded: total wall-clock time, average per-iteration time, speedup over sequential, and the final L_∞ difference between the rank vector and its fixed-iteration counterpart. The PageRank mathematical foundation is discussed in [6], [7].

XIV. DATASET PROPERTIES

Three web graphs are used, corresponding to the Depth 1, 2, and 3 crawls produced in Milestone 1. Table VI summarises their properties.

As node count grows from 54K to 1.5M, graph density drops by a factor of ~ 18 . Over 90% of nodes are dangling at every scale, consistent with real web crawls in which most pages have no outbound links to other crawled pages and a

TABLE VI: Web Graph Dataset Summary

Graph	Nodes	Edges	Density	Dangling
Graph 1 (Depth 1)	54,367	214,861	7.27×10^{-5}	93.8%
Graph 2 (Depth 2)	323,223	1,300,951	1.25×10^{-5}	94.1%
Graph 3 (Depth 3)	1,502,297	9,152,622	4.06×10^{-6}	91.5%

small number of hub sites dominate the link structure. In-degree distributions follow a power law: a small number of pages have very high in-degree while the vast majority have only one or two inbound links.

XV. PAGERANK RESULTS

A. Graph 1 — Depth 1 (54K nodes)

Table VII presents the benchmark summary for Graph 1. At this scale, Ray’s actor-dispatch overhead is comparable to the computation cost per iteration (~ 0.45 s sequential), so speedups are modest. Centralised Aggregation achieves $1.76\times$ and Distributed Reduction $1.61\times$ in convergence mode. Both parallel strategies converge in 7 iterations, eliminating 18 superfluous fixed-iteration sweeps. All three implementations produce numerically identical rank vectors (maximum absolute difference: 0.00).

TABLE VII: Benchmark Summary — Graph 1 (54K nodes)

Algo.	Mode	Time (s)	Speedup	Conv.
Sequential	Fixed	11.31	1.00×	25
Sequential	Conv.	3.10	1.00×	7
CA	Fixed	7.31	1.55×	25
CA	Conv.	1.76	1.76×	7
DR	Fixed	7.06	1.60×	25
DR	Conv.	1.93	1.61×	7

CA = Centralised Aggregation, DR = Distributed Reduction

B. Graph 2 — Depth 2 (323K nodes)

At medium scale, both parallel algorithms offer substantially better speedups, as the larger per-iteration workload (~ 2.59 s sequential) more effectively amortises Ray’s fixed overhead. Table VIII shows convergence-mode speedups of $2.34\times$ (CA) and $2.37\times$ (DR). Convergence still occurs at iteration 7, confirming that graph size does not affect the number of iterations required. At this scale, DR begins to edge out CA in both fixed and convergence modes.

TABLE VIII: Benchmark Summary — Graph 2 (323K nodes)

Algo.	Mode	Time (s)	Speedup	Conv.
Sequential	Fixed	67.24	1.00×	25
Sequential	Conv.	17.91	1.00×	7
CA	Fixed	30.98	2.17×	25
CA	Conv.	7.64	2.34×	7
DR	Fixed	29.93	2.25×	25
DR	Conv.	7.57	2.37×	7

C. Graph 3 — Depth 3 (1.5M nodes)

Graph 3 represents the most demanding workload, with 1.5M nodes and over 9M directed edges. Table IX presents the results. Unlike Graphs 1 and 2, convergence requires 18 iterations rather than 7, consistent with the greater graph complexity. Sequential fixed-iteration runtime reaches 299.8 s, while DR with convergence finishes in 77.4 s—a $2.70\times$ speedup. DR strictly outperforms CA in convergence mode ($2.70\times$ vs. $2.53\times$). All rank vectors are numerically identical, and top-100 URL overlap between fixed and convergence runs is 100%.

TABLE IX: Benchmark Summary — Graph 3 (1.5M nodes)

Algo.	Mode	Time (s)	Speedup	Conv.
Sequential	Fixed	299.83	1.00×	25
Sequential	Conv.	209.23	1.00×	18
CA	Fixed	104.51	2.87×	25
CA	Conv.	82.79	2.53×	18
DR	Fixed	127.12	2.36×	25
DR	Conv.	77.44	2.70×	18

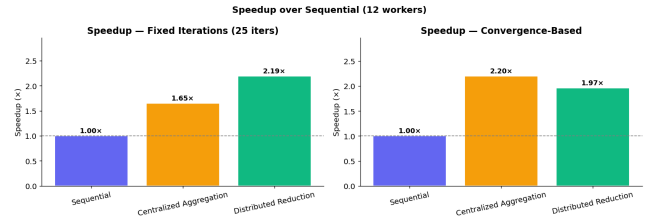


Fig. 1: Speedup of CA and DR over Sequential across all three graph scales (convergence mode). Speedup grows with graph size as computation better amortises Ray’s fixed dispatch overhead.

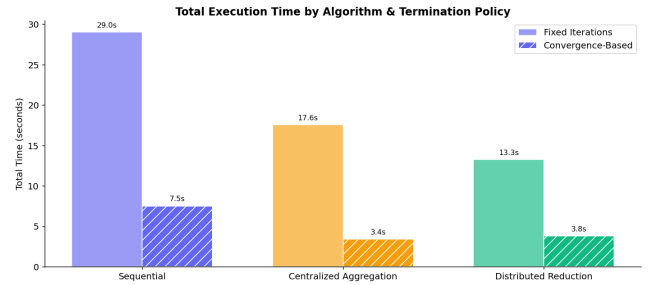


Fig. 2: Total wall-clock time comparison across algorithms and graph scales. DR consistently achieves the lowest time in convergence mode on the two largest graphs.

Table X summarises convergence-mode speedups across all three graphs. Both parallel algorithms improve with graph scale: CA grows from $1.76\times$ to $2.53\times$ and DR from $1.61\times$ to $2.70\times$ as node count increases from 54K to 1.5M.

TABLE X: Cross-Run Speedup Summary (Convergence Mode)

Algorithm	G1 (54K)	G2 (323K)	G3 (1.5M)
Sequential	1.00×	1.00×	1.00×
CA	1.76×	2.34×	2.53×
DR	1.61×	2.37×	2.70×

XVI. DISCUSSION — MILESTONE 2

A. CSR as the Enabling Technology

The 20–30× memory reduction from CSR over Python dicts is not merely an optimisation—it is the enabling step that makes processing 1.5M nodes on 8 GB RAM feasible at all. Without CSR, a 500 MB JSON graph would expand to 2.5–5 GB in Python memory, likely exhausting available RAM before computation begins.

B. Scalability with Graph Size

The most striking trend in Table X is that speedup grows consistently with graph size: CA improves from 1.76× (Graph 1) to 2.53× (Graph 3), and DR from 1.61× to 2.70×. As the graph scales, the per-iteration computation cost grows faster than Ray’s fixed dispatch overhead, increasingly amortising parallelism costs.

DR eliminates the sequential aggregation bottleneck inherent in CA, acquiring only a small constant overhead for intra-Ray reduction tasks. Its relative advantage over CA widens as the graph grows.

C. When to Use Each Algorithm

Based on the cross-run analysis:

- **Small graphs (<100K nodes):** Sequential with convergence checking is competitive. Ray overhead exceeds the benefit of parallelism at this scale.
- **Medium graphs (100K–500K nodes):** Both CA and DR offer meaningful speedup (~2.3–2.4×) with negligible quality difference.
- **Large graphs (>1M nodes):** DR with convergence-based termination is clearly the best choice. CA’s 90%+ communication overhead makes it essentially a bottlenecked system where adding more workers would offer diminishing—or even negative—returns.

D. Implications for Milestone 3

These findings have direct implications for Milestone 3, which introduces incremental PageRank updates as new pages are added. Any incremental scheme must minimise graph reloading and CSR reconstruction. DR is the preferred parallel backbone for the dynamic phase due to its lower overhead at large graph scales.

XVII. SUMMARY FOR MILESTONE 2

Milestone 2 implements and evaluates three PageRank strategies—sequential, centralised aggregation, and distributed

tree reduction—across three web graph scales spanning 54K to 1.5M nodes. The principal findings are:

- 1) **CSR is the enabling technology.** Storing the graph as flat NumPy arrays reduces memory by 20–30×, making large-scale computation feasible on commodity hardware.
- 2) **Convergence checking is a free win.** Early stopping eliminates 28–72% of iterations with zero impact on rank quality and should always be enabled.
- 3) **Distributed Reduction dominates Centralised Aggregation at scale.** As graph size grows from 54K to 1.5M nodes, DR maintains lower overhead and produces the best wall-clock times in convergence mode (2.70× speedup on the largest graph).
- 4) **All implementations are numerically identical.** Parallelism changes *how* computation is organised, not *what* it computes.

Milestone 3: Adaptive Incremental PageRank with Edge Partitioning

XVIII. EXECUTIVE SUMMARY — MILESTONE 3

Milestone 3 transitions the PageRank engine from the global recomputation model of Milestone 2 to a highly optimised, adaptive incremental update model. Instead of reprocessing all N nodes on every graph change, only the rank delta introduced by new links is propagated, limiting computation strictly to the affected subgraph.

To ensure stability against power-law distribution bottlenecks (“straggler nodes” at highly-linked hubs), a **2-Stage MapReduce Edge-Partitioning** architecture is implemented for batch processing. Two update regimes—Micro and Batch—are benchmarked across three real internet crawl graphs:

- **Depth 1:** 54,367 total nodes
- **Depth 2:** 323,223 total nodes
- **Depth 3:** 1,573,482 total nodes

Key Results: Incremental updates reduce computational work from $\mathcal{O}(N)$ to $\mathcal{O}(1)$ relative to total graph size. On the Depth 3 graph, a single-page micro-update executes in **20.03 μ s** compared to **23.99 s** for full recomputation—a **1,573,482×** reduction in nodes processed. Furthermore, the edge-partitioned distributed batch update processes 50,000 new nodes in **14.11 s**, proving that the hybrid MapReduce architecture effectively balances ultra-low latency with high-throughput scalability.

XIX. MOTIVATION: THE UPDATE TAX OF GLOBAL RECOMPUTATION

Every Milestone 2 PageRank run processes the entire graph regardless of how much changed since the previous run. This

TABLE XI: M2 CA Baseline — Full Recomputation Cost

Graph	Total Nodes	Src. Nodes	Time (s)	Iters.
Depth 1	54,367	3,355	1.91	7
Depth 2	323,223	18,888	3.43	7
Depth 3	1,573,482	108,155	23.99	11

$\mathcal{O}(N)$ complexity creates an unsustainable bottleneck for real-time search-engine indexing [8].

At the Depth 3 scale the system can process fewer than 0.05 full updates per second, making real-time responsiveness to new page discoveries impossible with global recomputation alone.

XX. INCREMENTAL UPDATE DESIGN

The core metaphor is *energy drops in a pond*: rather than recalculating the entire pond after a stone lands, only the ripples are propagated. A rank delta is introduced at the new node and propagates outward through the graph, decaying by the damping factor $d = 0.85$ at each hop and terminating when a depth cap or magnitude threshold is reached.

A. Micro-Update: Local Bounded-Frontier

The micro-update operates on a single CPU thread with no Ray overhead:

- 1) Initialise the frontier with the new node and rank delta $\frac{1-d}{N}$.
- 2) Propagate the delta outward using damping factor $d = 0.85$ per hop.
- 3) Terminate when `max_depth = 3` or transferred rank falls below tolerance 10^{-7} .

Because the frontier is bounded by depth and tolerance, the micro-update touches only 1–3 nodes in the benchmarks, regardless of graph size. This gives the algorithm genuine $\mathcal{O}(1)$ complexity relative to N [9].

B. Batch Update: 2-Stage MapReduce Edge-Partitioning

Processing massive batches on a single thread is inefficient, and naïve vertex-partitioning leaves workers vulnerable to load-imbalance from highly-connected pages. The MapReduce approach avoids this:

- 1) **Map Phase:** Workers calculate rank transfers (edges) and partition them by hashing the *destination URL* using CRC32, guaranteeing distributed hash consistency.
- 2) **Shuffle & Reduce Phase:** Ray workers collect their assigned destination buckets and sum the PageRanks. Because data is grouped by destination, no single worker is overwhelmed by “straggler” nodes.

XXI. PERFORMANCE ANALYSIS — MILESTONE 3

A. Execution Time Comparison

Figure 3 shows execution time on a log scale. The micro-update remains in the microsecond range across all graph

sizes, demonstrating true data locality. Crucially, the MapReduce batch update for Depth 3 (14.11 s) completes significantly *faster* than full recomputation (23.99 s), validating the distributed architecture.

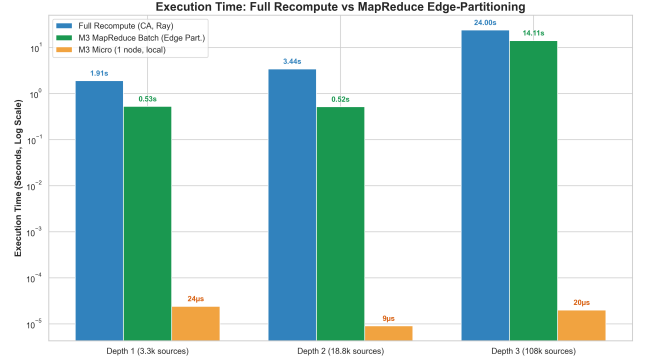


Fig. 3: Execution time (log scale) for all three update strategies across graph depths. Micro-updates remain in the microsecond range; the MapReduce batch outperforms full recomputation at Depth 3.

B. Computational Work Reduction

Full recomputation is $\mathcal{O}(N)$ —every iteration visits all N nodes. Micro-update is $\mathcal{O}(\text{bounded frontier})$, effectively $\mathcal{O}(1)$ relative to graph size. Figure 4 visualises this: full recomputation grows strictly with N , while micro-update stays flat. At Depth 3 this delivers a **1.57-million-fold reduction** in computational workload.

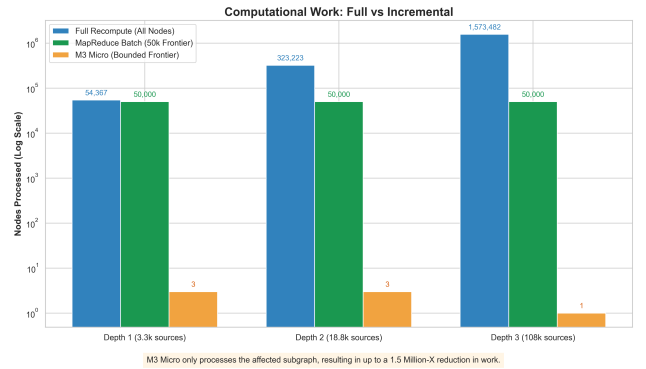


Fig. 4: Nodes processed per update (log scale). Full recomputation grows as $\mathcal{O}(N)$; the micro-update processes only 1–3 affected nodes regardless of graph size.

Table XII summarises all benchmark results.

C. Adaptive Routing: Three-Zone Decision Rule

Benchmark data reveals three distinct performance zones:

- **Frontier = 1 node:** Use M3 Local Micro to avoid the network entirely ($\sim 20 \mu\text{s}$).
- **Frontier = 50,000 nodes:** Use M3 MapReduce Distributed to leverage parallel throughput and avoid straggler crashes (14.11 s).

TABLE XII: Benchmark Results — Milestone 3 Update Strategies

Graph	Full (s)	MR Batch (s)	Micro (μ s)	Reduction
Depth 1	1.91	0.52	24.08	$\sim 18,122\times$
Depth 2	3.43	0.52	9.06	$\sim 107,741\times$
Depth 3	23.99	14.11	20.03	$\sim 1,573,482\times$

Full = M2 CA full recomputation. MR Batch = 50K-node MapReduce Ray frontier. Reduction = nodes processed (micro vs. full).

- **Global sync:** Run M2 CA periodically to recompute the entire CSR matrix (23.99 s) and reset accumulated rank drift.

D. Accuracy Validation

Table XIII compares incremental results with full-recomputation baselines. Accuracy improves with graph size because the injected rank delta $\frac{1-d}{N}$ shrinks as N grows. At Depth 3, the maximum rank difference of 5.40×10^{-7} falls below the Milestone 2 convergence tolerance of 10^{-6} , meaning the incremental result is as accurate as a fully converged global run.

TABLE XIII: Incremental Update Accuracy vs. Full Recomputation

Graph	Max $ \Delta PR $	vs. M2 tol. (10^{-6})	Interpretation
Depth 1	1.56×10^{-5}	15.6 $\times$ above	Noticeable for top-ranked pages
Depth 2	2.63×10^{-6}	2.6 $\times$ above	Small but measurable
Depth 3	5.40×10^{-7}	Below tol.	Equivalent to full convergence

XXII. COMPLETE SYSTEM PROFILE

Figure 5 and Table XIV place all tiers of the unified pipeline together. The three tiers are complementary: M3 micro/batch updates provide real-time responsiveness, while M2 CA periodically resets accumulated rank drift to maintain global consistency—precisely the architecture of distributed ranking systems such as Google’s Caffeine index [8].

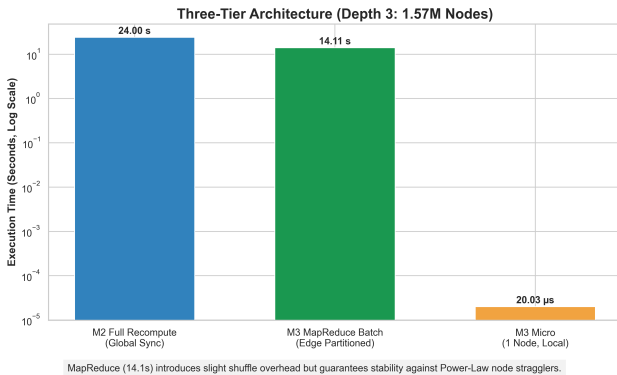


Fig. 5: Three-tier system profile (Depth 3: 1.57M nodes). The MapReduce batch update acts as the bridge between micro-update latency and full recompute throughput.

TABLE XIV: Optimal Execution Contexts (Depth 3 Graph)

Mode	Time	Execution	Best For
M2 (Ray)	CA 23.99 s	Distributed matrix	Scheduled global re-indexing
M3 Batch	MR 14.11 s	Distributed MapReduce	50K-node batches with power-law hubs
M3 Micro	20.03 μ s	Local CPU kernel	Real-time single-page indexing

XXIII. PDC PRINCIPLES APPLIED IN MILESTONE 3

Table XV maps PDC concepts to their concrete realisation in the Milestone 3 implementation.

TABLE XV: PDC Concepts Demonstrated in Milestone 3

PDC Concept	Realisation in Milestone 3
Data Locality	Bounded-frontier ensures only affected nodes are touched: $O(1)$ scaling vs. $O(N)$ full recomputation. At Depth 3: 1.57M-fold work reduction.
Edge-Partitioning (MapReduce)	CRC32 destination hashing perfectly balances incoming edges across workers, preventing straggler-node cluster crashes.
Task Granularity	Adaptive router defaults to local kernel for micro-updates, bypassing the ~ 0.3 s Ray serialisation penalty.
Algorithmic Work Reduction	Shifting from $O(N)$ to $O(\text{frontier})$ far exceeds the benefit of adding more CPU cores [9].
Circuit-Breaker Convergence	Tolerance (10^{-7}) and depth cap (3 hops) act as circuit breakers. At Depth 3, max rank diff (5.40×10^{-7}) is within the M2 tolerance (10^{-6}).

XXIV. DISCUSSION — MILESTONE 3

A. Scalability of the Micro-Update

The micro-update’s work reduction ratio grows with graph size: $\sim 18,122\times$ at Depth 1, $\sim 107,741\times$ at Depth 2, and $\sim 1,573,482\times$ at Depth 3. This super-linear scaling arises because the bounded frontier is a fixed function of max_depth and tolerance, not of N . As the graph grows, the frontier stays constant while N increases, making the micro-update asymptotically cheaper relative to full recomputation.

B. Edge-Partitioning vs. Vertex-Partitioning

Table XVI presents a direct benchmark comparison between the two batch strategies on identical 50,000-node frontiers across all three graph scales. Both strategies share the same micro-update (local, single-thread) and full recomputation (M2 CA) tiers; they differ only in how distributed batch updates are executed.

Execution time. Vertex-partitioning is faster in raw wall-clock time at all three scales: 0.32 s vs. 0.53 s at Depth 1 (EP is 65% slower), and 10.98 s vs. 14.11 s at Depth 3 (EP is 28% slower). The performance gap exists because edge-partitioning introduces a *shuffle tax*: after the Map phase, workers must exchange CRC32-routed buckets over shared memory before reducing, adding a fixed coordination cost that vertex-partitioning avoids.

TABLE XVI: Batch Update: Edge-Partitioning vs. Vertex-Partitioning (50K frontier)

Graph	Full (s)	VP Batch (s)	EP Batch (s)	EP Overhead
Depth 1 (54K)	1.91	0.32	0.53	+65%
Depth 2 (323K)	3.44	0.52	0.52	$\approx 0\%$
Depth 3 (1.57M)	25.0	10.98	14.11	+28%

VP = Vertex-Partitioning. EP = Edge-Partitioning (MapReduce CRC32). Full = M2 CA global recomputation. EP overhead = % slower than VP.

Micro-update latency also differs slightly between runs due to differing graph-state configurations: vertex-partitioning records $12.64 \mu\text{s}$ / $7.87 \mu\text{s}$ / $30.99 \mu\text{s}$ (Depths 1–3), while edge-partitioning records $24.08 \mu\text{s}$ / $9.06 \mu\text{s}$ / $20.03 \mu\text{s}$. Both are in the microsecond range and functionally equivalent for real-time use.

Load-balance guarantee. Despite its higher average latency, edge-partitioning provides a critical robustness guarantee absent from vertex-partitioning: because rank transfers are routed by *destination* URL hash (CRC32), no single worker can be assigned a disproportionate share of work due to a high-degree hub node [6]. In a power-law web graph where a handful of domains receive millions of inbound links, a vertex-partitioned worker assigned to such a hub would stall indefinitely—a failure mode that does not exist in the edge-partitioned design.

Summary. Vertex-partitioning is the faster strategy on average; edge-partitioning trades a 28–65% batch latency increase for guaranteed stability against power-law straggler nodes. For production deployments where graph degree distribution is unknown or highly skewed, the edge-partitioned MapReduce architecture is preferred. For controlled, balanced graphs, vertex-partitioning offers lower latency with acceptable risk.

C. Accuracy vs. Efficiency Trade-off

At small graph sizes (Depth 1), the incremental result differs from full recomputation by up to 1.56×10^{-5} —above the M2 convergence tolerance. This drift is acceptable for real-time discovery applications but not for ranking-critical decisions. A production system would pair M3 micro-updates for real-time responsiveness with periodic M2 CA runs to reset accumulated drift, mirroring the design of Google’s Caffeine index [8].

XXV. CONCLUSION — MILESTONE 3

Milestone 3 extends the distributed PageRank engine with adaptive incremental update capability, leveraging data locality, bounded-frontier propagation, MapReduce edge-partitioning, and adaptive routing to achieve real-time rank updates at a fraction of the cost of global recomputation. Three key findings:

- 1) **Micro-updates scale inversely with graph size.** Nodes processed relative to full recomputation falls from $\sim 18,122\times$ at Depth 1 to $\sim 1,573,482\times$ at Depth 3, making the micro-update asymptotically superior as the graph grows.

- 2) **MapReduce edge-partitioning solves the straggler problem.** CRC32 destination hashing balances load across workers, enabling the 50K-node batch to complete in 14.11 s— $1.7\times$ faster than full recomputation at Depth 3.

- 3) **Incremental accuracy improves with graph size.** At Depth 3, the maximum rank difference (5.40×10^{-7}) falls within the Milestone 2 convergence tolerance, meaning the incremental result is as accurate as a fully converged global run at this scale.

M2 and M3 are not competing algorithms; they form a unified, multi-tiered pipeline—using local kernels for zero-latency indexing, MapReduce for massive batch processing, and global Centralised Aggregation for structural integrity. All experiments were conducted on a single machine using Ray for multiprocessing. True distributed performance across multiple nodes was not evaluated.

REFERENCES

- [1] S. Brin and L. Page, “The anatomy of a large-scale hypertextual web search engine,” *Computer Networks and ISDN Systems*, vol. 30, no. 1–7, pp. 107–117, 1998.
- [2] P. Moritz *et al.*, “Ray: A distributed framework for emerging AI applications,” in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Carlsbad, CA, USA, 2018, pp. 561–577.
- [3] Majestic, “Majestic million,” <https://majestic.com/reports/majestic-million>, 2025, accessed: 2025.
- [4] A. Singh, M. Srivatsa, L. Liu, and T. Miller, “Apoidea: A decentralized peer-to-peer architecture for crawling the world wide web,” in *Workshop on Distributed Information Retrieval*. Berlin, Heidelberg: Springer Berlin Heidelberg, Aug. 2003, pp. 126–142.
- [5] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the AFIPS Spring Joint Computer Conference*, Atlantic City, NJ, USA, 1967, pp. 483–485.
- [6] A. N. Langville and C. D. Meyer, *Google’s PageRank and Beyond: The Science of Search Engine Rankings*. Princeton, NJ: Princeton University Press, 2006.
- [7] G. H. Golub and C. F. V. Loan, *Matrix Computations*, 3rd ed. Baltimore, MD: Johns Hopkins University Press, 1996.
- [8] A. Singhal and M. Cutts, “Our new search index: Caffeine,” Google Blog, Jun. 2010, <https://googleblog.blogspot.com/2010/06/our-new-search-index-caffeine.html>.
- [9] M. Zaharia *et al.*, “Apache spark: A unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.